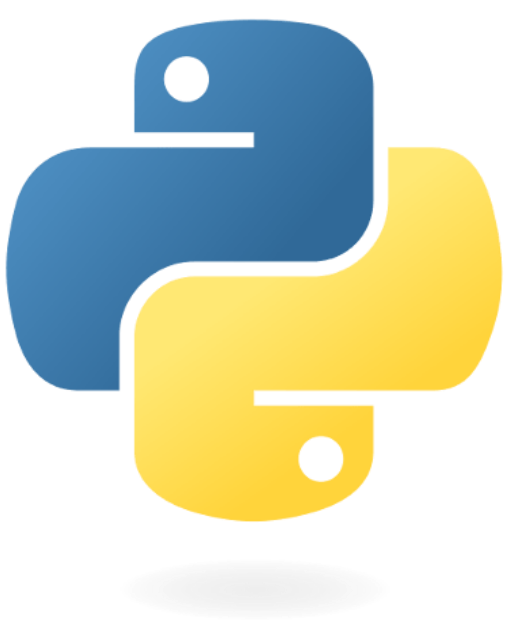


python™

Prof. Luciano Nunes
Jun/2018
lnunes at gmail

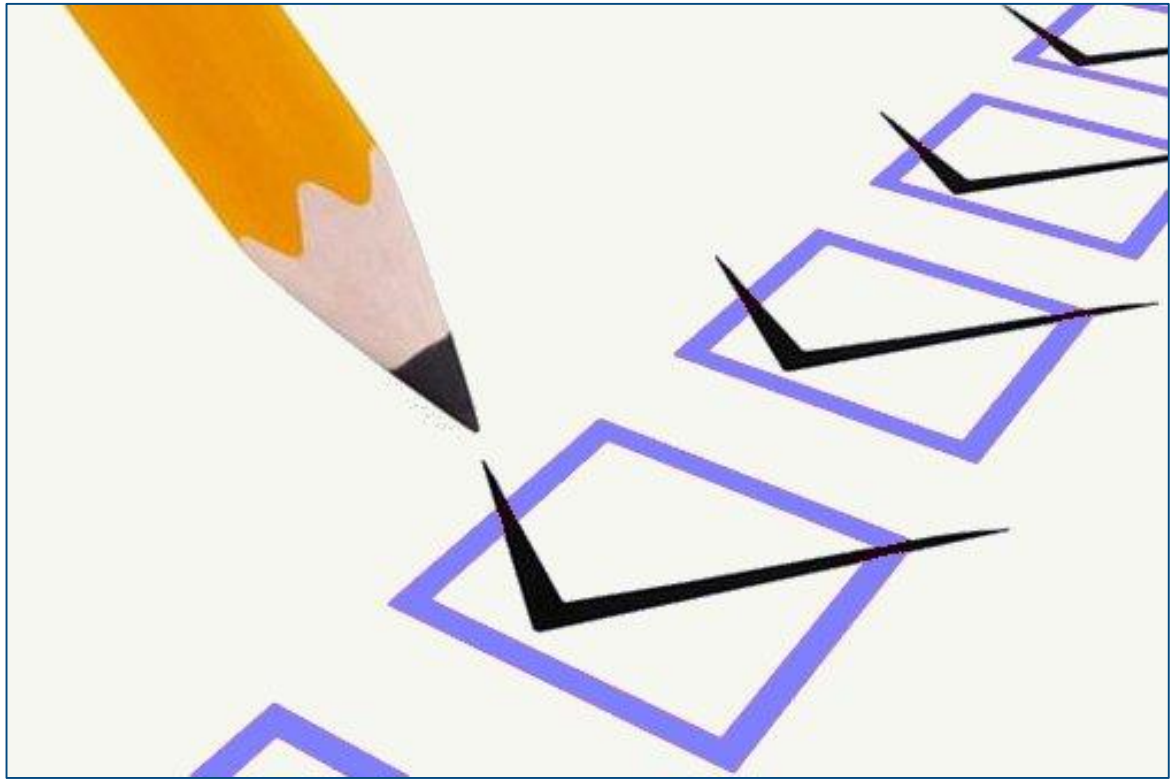
Objetivos → Abordagem

- Desenvolver raciocínio lógico e capacidade de escrever algoritmos utilizando a linguagem de programação Python
- Conhecer estruturas de dados disponibilizadas pela linguagem e escolher as mais convenientes para solução de problemas diversos
- Aprender a ler e escrever em arquivos e utilizar expressões e programação multithread quando necessário.

- Apresentação da teoria de forma gradual e ordenada passando pelos tipos, estruturas de dados, controle de fluxo, laços, arquivos e tratamento de exceções
- Demonstração com exemplos práticos de cada tópico apresentado
- Aplicação prática na forma de exercícios a serem resolvidos, exigindo o emprego de raciocício lógico e conhecimentos adquiridos da linguagem

Critérios de Avaliação

- Ao final de cada aula, será disponibilizada uma lista de exercícios práticos
- Cada lista de exercícios valerá 2,5 (dois pontos e meio)
- As listas deverão ser entregues até o início da aula subsequente
- Listas entregues após o prazo determinado, valerão 1,00 (um ponto) apenas
- Na última aula do módulo, será aplicada uma avaliação prática, valendo 5 (cinco pontos)
- A nota final do módulo será a somatória da pontuação de cada lista de exercícios com a pontuação da avaliação prática



Motive-se

- Defina objetivos (metas e prazos) e periodicamente verifique os resultados. Faça ajustes quando necessário
- Aplique os conhecimentos adquiridos neste curso no seu cotidiano
- Se desafie, tente algo além do proposto
- Trabalhe de forma colaborativa, isso o ajudará a melhorar seu desempenho
- Pratique. Quando terminar, pratique mais.
- As oportunidades normalmente se apresentam disfarçadas de trabalho árduo e é por isso que muitos não as reconhecem (Bernardinho)
- E por último e não menos importante, divirta-se sempre!



DiD YOU KNOW?

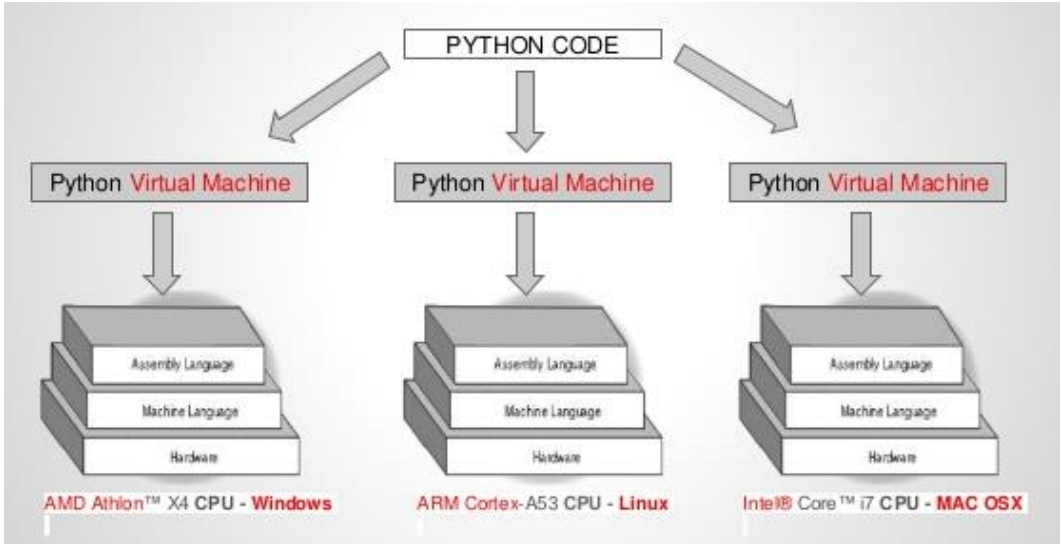


Linguagens de Programação

- Baixo Nível
  - Código Binário ou linguagem de máquina
  - Assembly

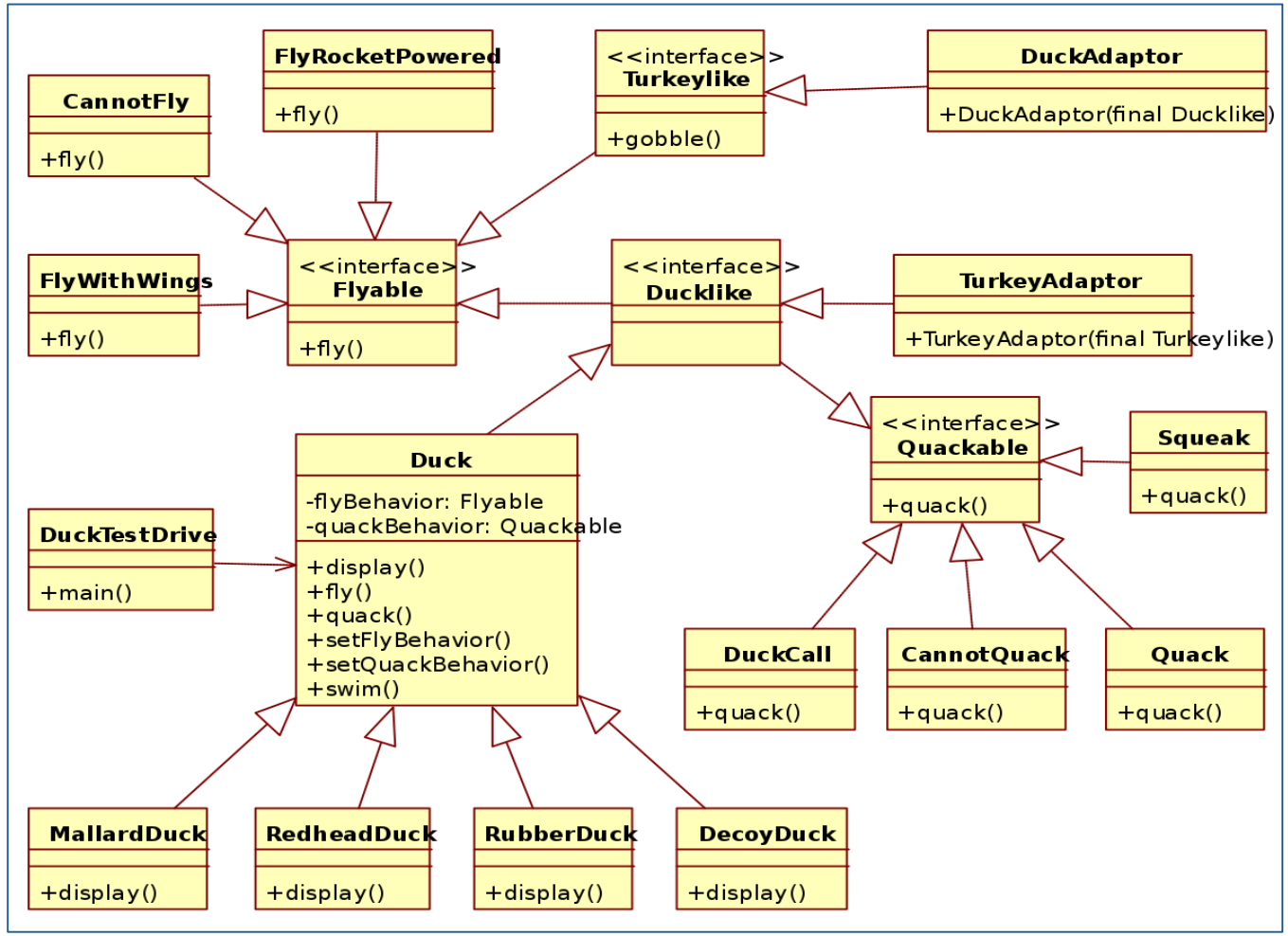
- Alto Nível
  - Compiladas
    - C / C++
    - Pascal
    - Delphi
  - Interpretadas
    - Java
    - C#
    - PHP
    - Javascript
    - Python

- Código Intermediário
  - Bytecode
  - JIT Compilation
  - Virtual Machine



A Importância da Análise Orientada à Objetos

- Abstração
- Polimorfismo
- Herança
- Acoplamento
- Coesão
- Reutilização
- UML
- Saber construir e interpretar diagramas (Casos de Uso, Classes, Sequencia)



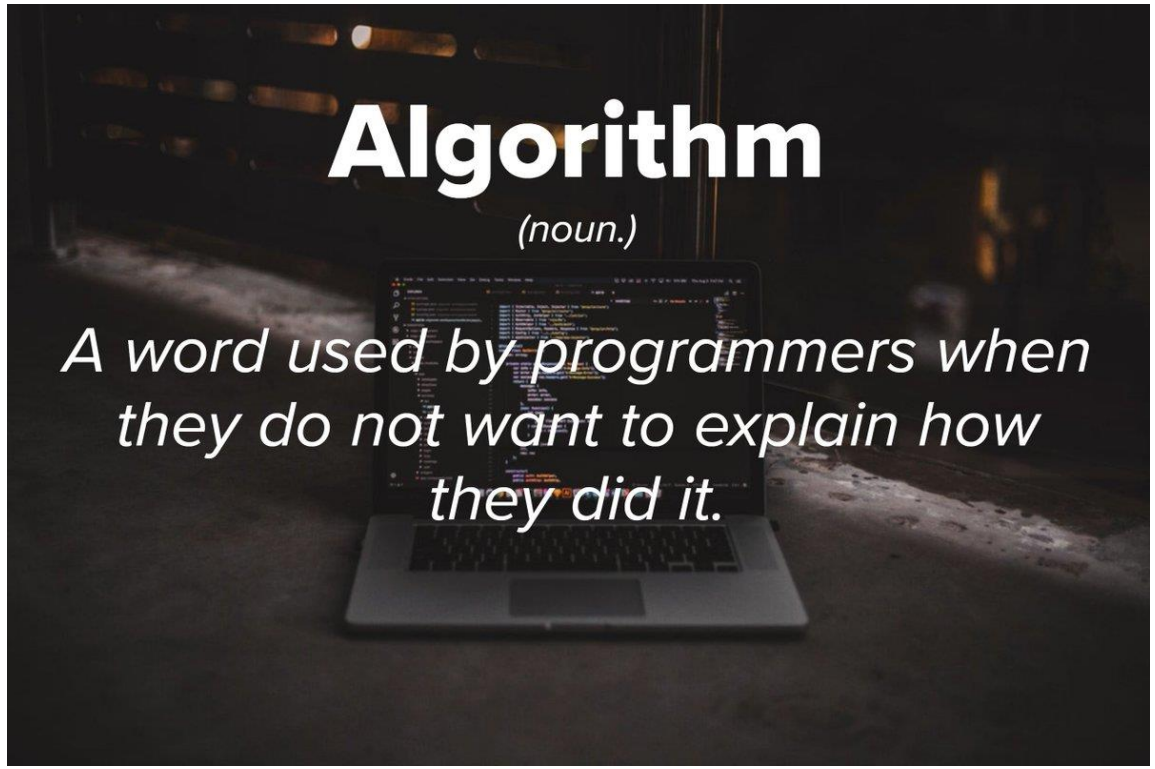
Boas Práticas de Programação

- Code Conventions
  - https://www.python.org/dev/peps/pep-0008/
- Clean Code
  - Fácil de entender
  - Direto ao ponto
  - Eficiente
  - Elegante
- Nomes Significativos (substantivos e verbos)
- Documentação
- Comentários
- Testes Unítários
- Code Review

- Dicas
  - Antes de iniciar a programação tenha certeza do entendimento do problema
  - Adquira o hábito de comentar seu código no momento em que estiver programando, preferenciamente antes de escrever a instrução
  - Seja breve e direto nos comentários, evitando escrever o que não é relevante
  - Dê preferência aos comentários em inglês, já imaginou ler um código com comentários em Latin? ☺
  - Não exagere, pois comentários em excesso podem atrapalhar ao invés de ajudar, procure comentar trechos de código

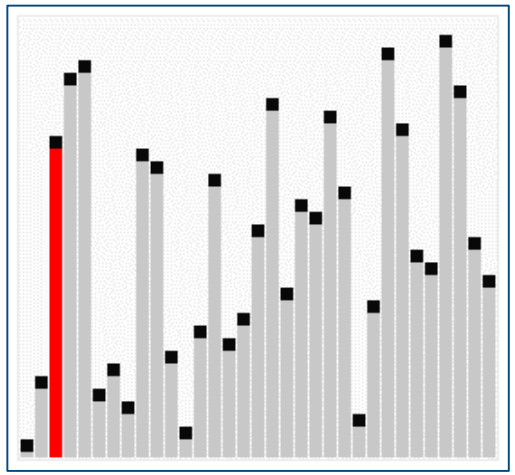
O que é um Algoritmo?

- Algoritmo é uma sequência finita de instruções bem definidas e não ambíguas, cada uma das quais devendo ser executadas mecânica ou eletronicamente em um intervalo de tempo finito e com uma quantidade de esforço finita. (Wikipedia)
- Simplificadamente um algoritmo é uma receita, um conjunto de instruções bem definidas para solucionar um problema conhecido.
- Dica: Algoritmo não se aprende copiando ou estudando algoritmos prontos e sim construindo e testando seus próprios algoritmos



Algoritmo BubbleSort

- Percorra o vetor inteiro comparando elementos adjacentes (dois a dois)
- Troque as posições dos elements se eles estiverem fora de ordem
- Repita os dois passos acima com os primeiros n-1 itens, depois com os primeiros n-2 itens, até que reste apenas um item



BubbleSort.py

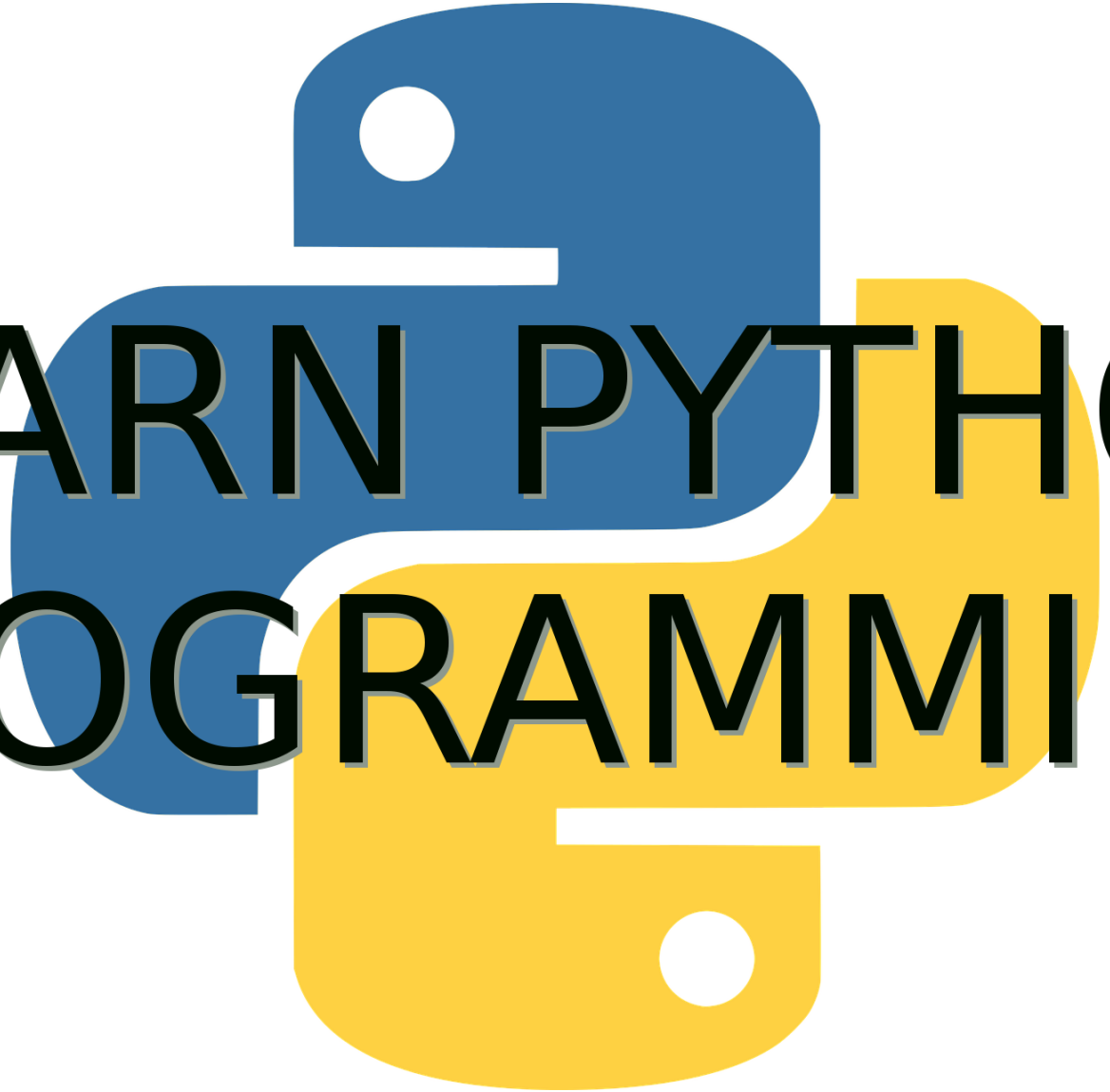
```
1   def bubbleSort(alist):
2       for passnum in range(len(alist) - 1, 0, -1):
3           for i in range(passnum):
4               if alist[i] > alist[i + 1]:
5                   temp = alist[i]
6                   alist[i] = alist[i + 1]
7                   alist[i + 1] = temp
8
9   alist = [111, 245, 54,26,93,17,77,31,44,55,20]
10  bubbleSort (alist)
11  print (alist)
12
```

BubbleSort

```
C:\dev\workspace_git\DataScienceFacens\HelloWorldProje
[17, 20, 26, 31, 44, 54, 55, 77, 93, 111, 245]

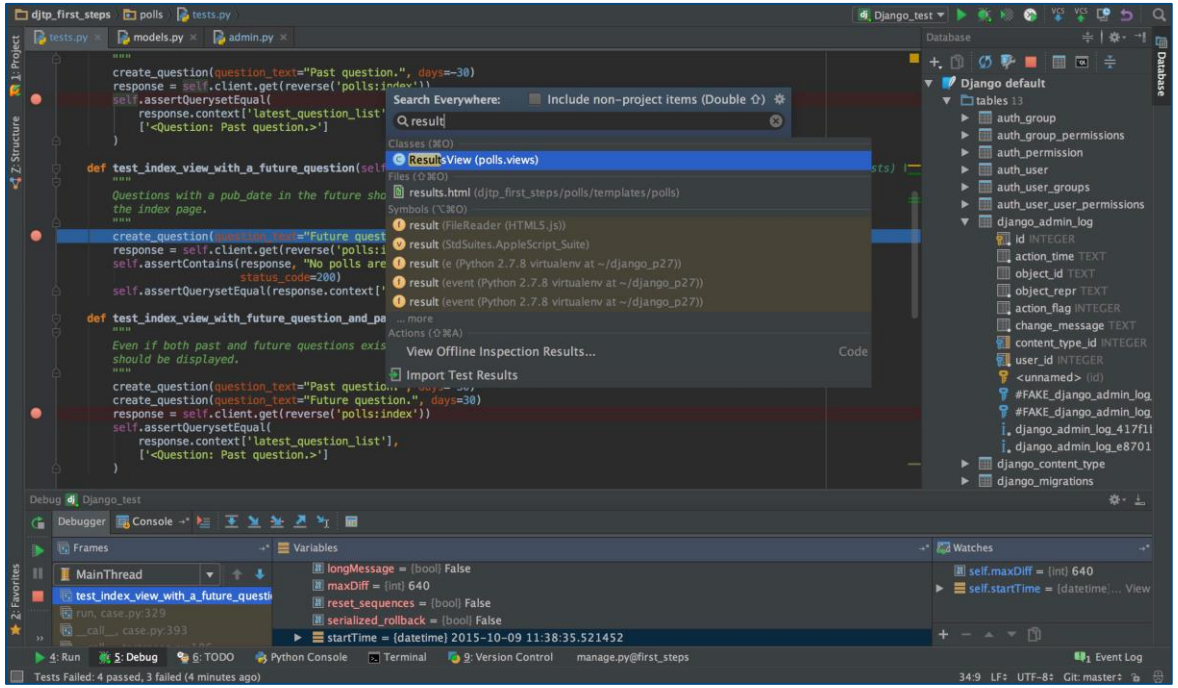
Process finished with exit code 0
```


PRACTICE
MAKES
PERFECT


LEARN PYTHON
PROGRAMMING

Ambiente de Desenvolvimento

- IDE Versus Ambiente de desenvolvimento
- Aspectos a serem considerados
  - Estrutura e organização dos códigos fontes
  - Compilador e executor integrado
  - Ferramenta para depuração (debug)
  - Ferramenta para inspeção (variable inspect)
  - Gerenciador de pacotes
  - Consumo de recursos
- Ambientes de Desenvolvimento para Python
  - PyCharm
  - IDLE
  - Atom
  - Jupyter Notebook
  - Anaconda
  - Visual Studio Code



https://www.jetbrains.com/pycharm/

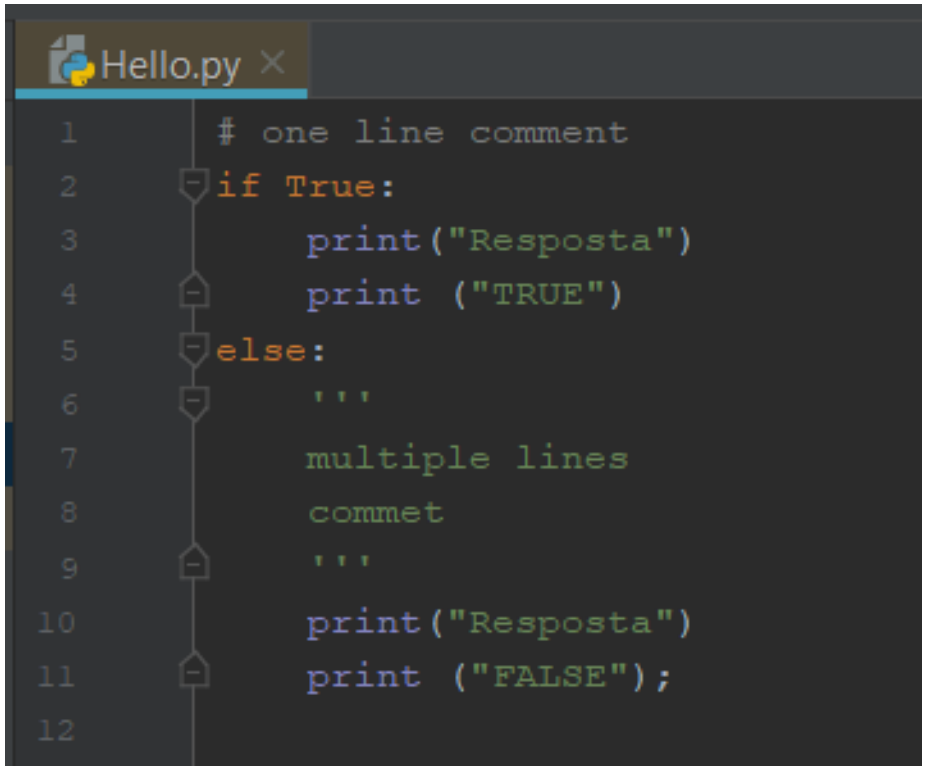
Uma Breve Introdução

- Python é uma linguagem de programação desenvolvida por Guido van Rossum no final da década de 1980 com o objetivo de ser fácil e intuitiva, porém poderosa.
- Suas principais características são:
  - Linguagem de programação de alto nível
  - Interpretada e de código-fonte aberto
  - Interativa
  - Multi-plataforma e Multi-paradigma
  - Sintaxe simples, fácil de aprender e de manter
  - Tipagem forte e dinâmica
  - Tudo em Python é um objeto: variáveis, funções, etc. Cada objeto tem um ID, tipo e valor



Sintaxe Básica

- A identação é usada em Python para delimitar blocos de código. O número de espaços pode variar, porem todos os *statements* (instruções) dentro de um mesmo bloco, devem possuir a mesma quantidade de espaços
- A primeira linha que compõe *statements* como *if*, *else*, *while*, *def*, e *class* deve ser finalizada com dois pontos :
- O ponto e vírgula é opcional ao final de um *statement*
- Comentários de apenas uma linha começam por # e de múltiplas linhas por ’’’
- Arquivos de programas Python tem extensão .py



Variáveis

- Python é dinamicamente tipado, você não precisa declarar as variáveis
- A declaração acontece automaticamente no momento em que um valor é atribuído à variável
- Variáveis podem mudar de tipo, simplesmente com uma nova atribuição de um tipo diferente
- Python permite atribuir um único valor à várias variáveis ao mesmo tempo
- Assim como permite atribuir multiplos objetos à múltiplas variáveis

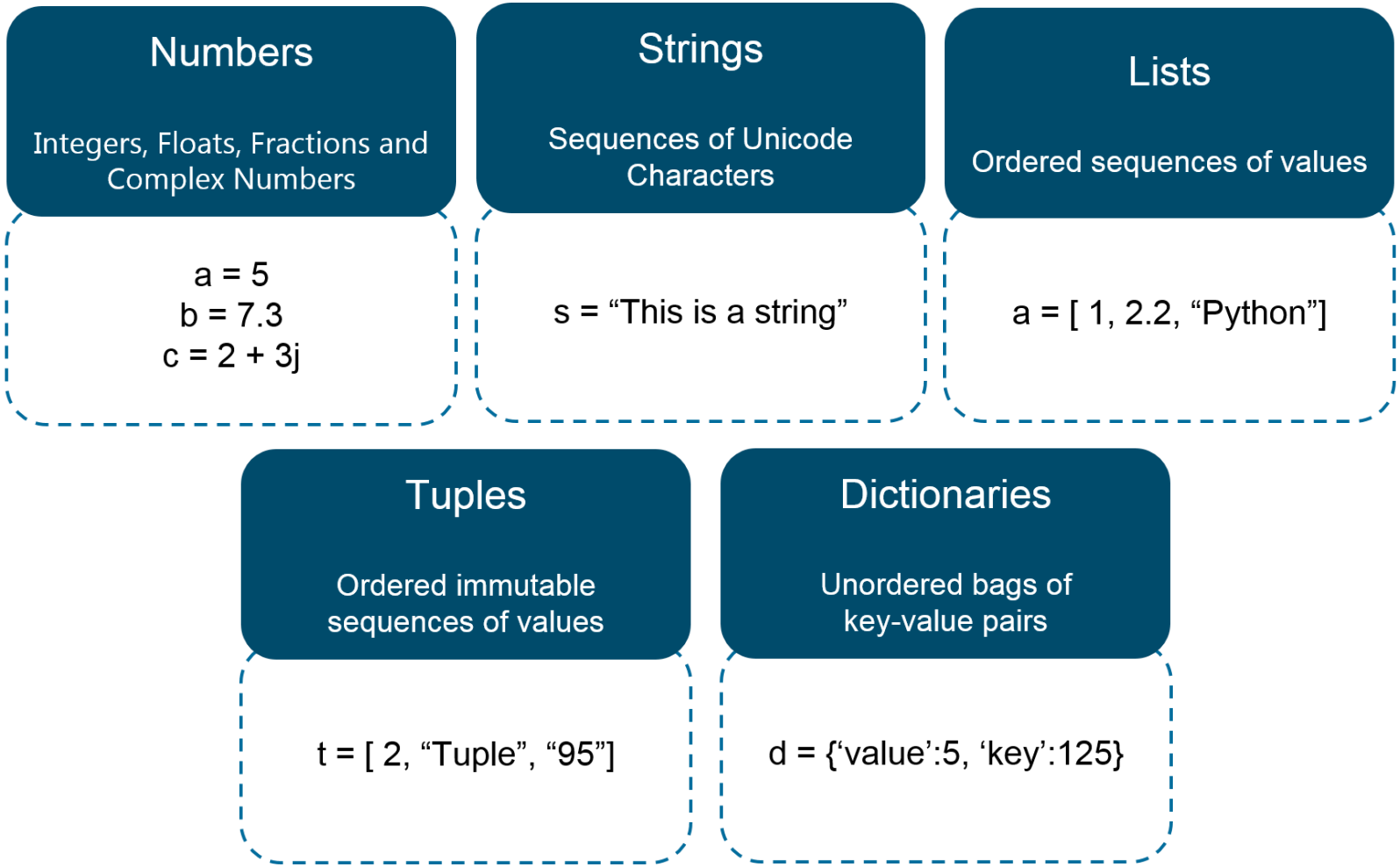
```
max = 10                # integer
value = 150.0           # float point
name = "Python"         # string
nothing = None          # null value
```

```
x = 1
x = "String value"
```

```
a = b = c = 1
```

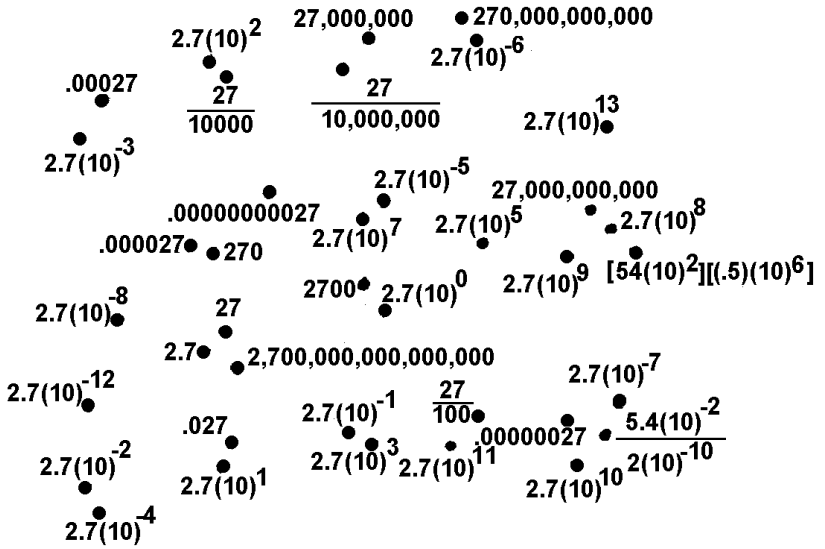
```
a, b, c = 1, 2, "Python"
```

Tipos de Dados



Tipos Numericos

- Números são objetos imutáveis em Python, isso significa que seus valores não podem ser alterados
- Existem 3 tipos de dados para números em Python
  - int (signed integer)
    - Também chamado apenas de integer, comporta números positivos e negativos sem casas decimais
  - float (floating point)
    - Representa números reais e são escritos com ponto decimal, dividindo um inteiro em partes fracionárias, também podendo ser escrito em notação científica com “e” indicando potencia de 10 (ex: 2.5e2)
  - complex
    - São escritos por dois valores reais, a parte real e a parte imaginária na forma (real + IMAG J). Neste caso o número i ($\sqrt{-1}$) é designado pela letra j. (Não são muito utilizados)



Funções mais comuns com tipos numéricos

Função	Descrição
int (x)	Converte x em um inteiro
float (x)	Converte x em um ponto-flutuante
abs (x)	Retorna o valor absoluto de x
exp (x)	Retorna o exponencial de x (e^x)
log (x)	Retorna o logaritmo natural de x (inverso da função exponencial)
pow (x, y)	Retorna o valor de x elevado à potencia y
sqrt (x)	Retorna a raiz quadrada de x
round (x, y)	Retorna x arredondado em y casas decimais

Strings

- String, assim como numbers, são objetos imutáveis em Python. Dessa forma um update só pode ser possível com a alocação de um novo objeto com o novo conteúdo
- Python não suporta tipos “char”. Um char em Python é considerado uma String de tamanho 1
- Uma String pode ser atribuída de várias formas diferentes
  - ‘Uma String’
  - “Outro exemplo de String”
  - ’’’String com
    multiplas linhas’’’
- Strings iniciam sempre com indice 0
- Pode se usar operações como slicing ([], [:]), concatenação (+), repetição (*) e *membership* (in)



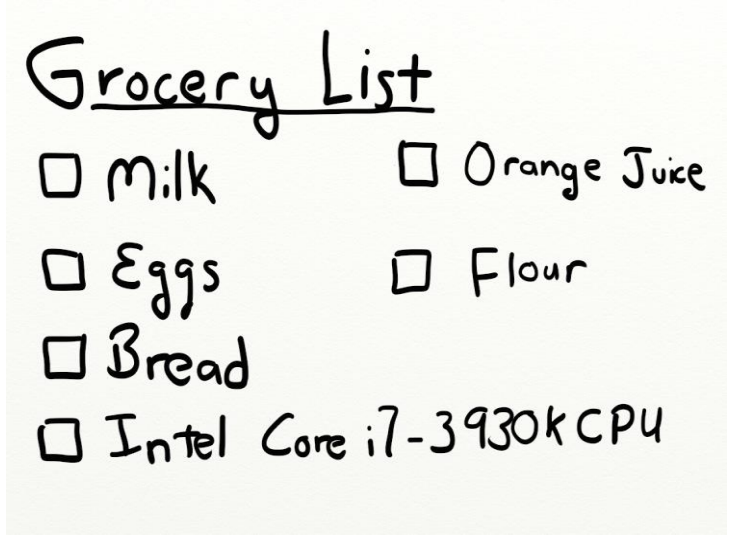
Funções e métodos mais comuns com tipo String

Função	Descrição
str (num)	Converte um número em String
len (str)	Retorna o tamanho de uma String
str.count (s)	Retorna a quatidade de conjuntos *s* presentes na string
str.isalpha ()	Retorna False se a string contiver algum caracter que não seja letras
str.isdigit ()	Retorna False se a string contiver algum caracter que não seja número
str.lower ()	Retorna a string transformada em minúsculos
str.upper ()	Retorna a string transformada em maiúsculos
str.replace (old, new)	Substitui uma porção da string por outro conteúdo
str.strip ()	Retira espaços em branco no começo e no fim da string
str.title ()	Retorna a string capitalizada (iniciais em maiúscula)
str.split (delimit)	Separa uma string conforme um delimitador. É o inverso do join()
str.join (sequence)	Junta cada item da string com um delimitador especificado. É o inverso do split().

Listas

- Listas são grupos de itens ou elementos indexados, não necessariamente do mesmo tipo
- Listas são mutáveis, isso significa que seu conteúdo pode ser alterado sem que um novo objeto seja criado
- Para atribuir elementos a uma lista é necessário que estejam entre colchetes e separados por vírgula, ex:

  l = [item1, item2, ... Itemn]
- Os indices em uma lista sempre se iniciam em 0 (zero)
- Listas podem conter sublistas que podem conter sublistas e assim por diante, tornando esse tipo de dado extremamente versátil
- Pode se usar operações como slicing ([], [:]), concatenação (+), repetição (*) e *membership* (in)



Funções e métodos mais comuns com tipo Lista

Função	Descrição
len(lista)	Retorna o tamanho total da lista
max(lista)	Retorna o maior elemento da lista
min(lista)	Retorna o menor elemento da lista
list(tupla)	Converte uma tupla em uma lista
list.append(obj)	Adiciona um objeto à lista
list.insert(index, obj)	Insere um objeto à lista em determinada posição
list.count(obj)	Retorna a quantidade de vezes que um determinado objeto ocorre na lista
list.index(obj)	Retorna o primeiro indice de ocorrencia de um determinado objeto
list.remove(obj)	Remove um objeto da lista
list.reverse()	Reverta o ordenamento da lista
list.sort()	Ordena a lista

Tuplas

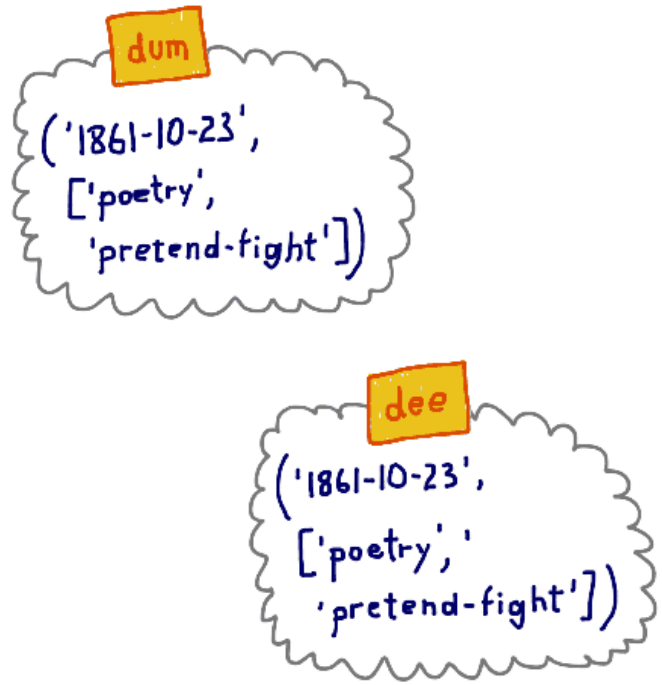
- Tuplas são tipos muito semelhantes às listas, porém imutáveis
- Por esse motivo, tuplas são mais eficientes que as listas, dessa forma quando não existe a necessidade de alteração de seu conteúdo, devem ser escolhidas preferencialmente
- Como são imutávais, protegem os dados contidos nela de alterações acidentais
- Para atribuir elementos a uma tupla é necessário que estejam entre parêntesis e separados por vírgula, ex:

  t = (item1, item2, ... Itemn)

- Para tuplas de um elemento apenas é necessário incluir a vírgula após o elemento

  t = (item1,)

- Indices funcionam da mesma forma das listas, assim como os métodos e funções



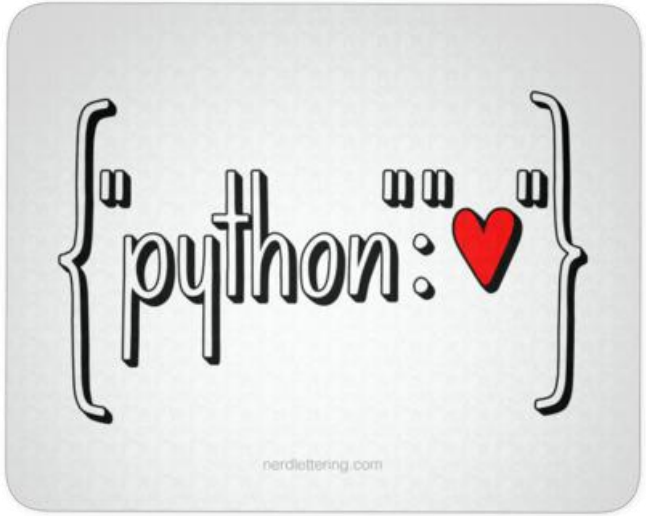
Dicionários

- Dicionários são uma implementação de hash table que consiste em uma lista de pares de chave-valor, sem ordenação
- Chaves, devem ser tipos imutáveis e usualmente são utilizadas strings ou números
- Valores podem ser qualquer tipo de objeto Python
- Para atribuir elementos a um dicionário é necessário que estejam entre chaves, separados por vírgula, e com os pares chave, valor separados por : (dois pontos)

  dic = {‘k1’:1, ‘k2’:10, ... ‘kn’:n}

- Os elementos de um dicionário podem ser acessados ou alterados através de sua chave entre colchetes

  dic[‘k1’] = 2

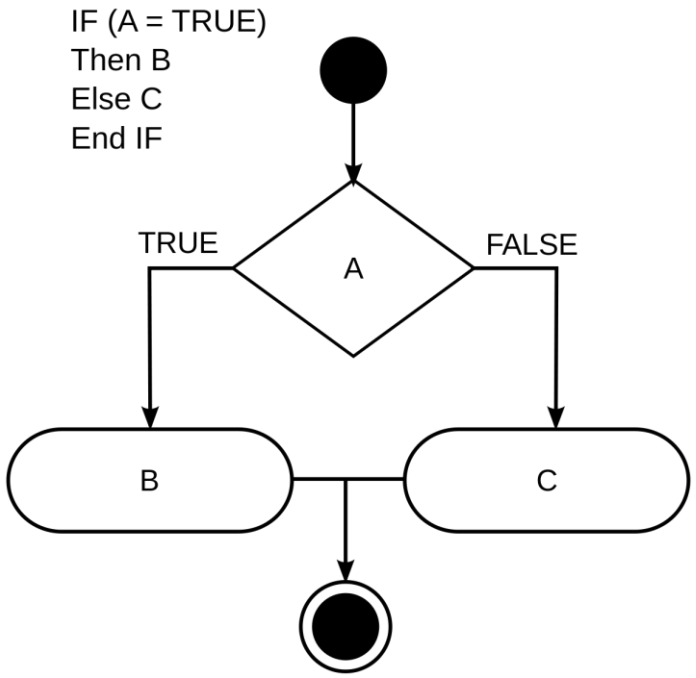


Funções métodos mais comuns com tipo Dicionário

Função	Descrição
len(dict)	Retorna o número de elementos do dicionário
str(dict)	Retorna a representação em formato string do dicionário
dict.keys()	Retorna a lista de chaves do dicionário
dict.values()	Retorna a lista de valores do dicionário
dict.items()	Retorna a lista de chave, valor do dicionário
dict.get(key, default=None)	Retorna o valor de uma chave ou um valor default caso não seja encontrada
dict.update(dict2)	Insere um elemento (chave, valor) no dicionário
dict.clear()	Remove todos os elementos do diciónário

Estruturas de Controle de Fluxo

- Podemos dividir as estruturas de controle de fluxo em 2 grupos
  - Estruturas de Seleção
  - Estruturas de Repetição
- Estrutura de Seleção são utilizadas para decidir qual fluxo de execução deverá ser tomado pelo programa, dada uma determinada condição ou um conjunto de condições
- São representadas em Python, assim como em outras linguagens pelas instruções:
  - if
  - elif
  - else



Estruturas de Controle de Fluxo

- Estruturas de Seleção sempre utilizam *expressões booleanas* para representar uma determinada condição
- True e False são Objetos imutáveis da classe *bool* e assume-se que qualquer valor diferente de 0 (zero) E diferente de null são considerados True, todos os outros valores considerados False
- Estruturas de Seleção podem ser aninhadas e combinadas juntamente com operadores (lógicos, aritméticos e relacionais), podendo tornar-se complexas estruturas de decisão

Selection.py

```
var1 = 100
if var1:
    print(var1)
```

Selection

```
C:\dev\workspace_git\DataScienceFac
100

Process finished with exit code 0
```

Selection.py

```
var1 = 100
if var1:
    print('OK')
else:
    print('NOK')
```

Selection

```
C:\dev\workspace_git\DataScienceFa
OK

Process finished with exit code 0
```

Selection.py

```
var = 100
if var < 150:
    if var == 150:
        print("150")
    elif var == 100:
        print("100")
    elif var <= 50:
        print("<= 50")
else:
    print(">= 150")
```

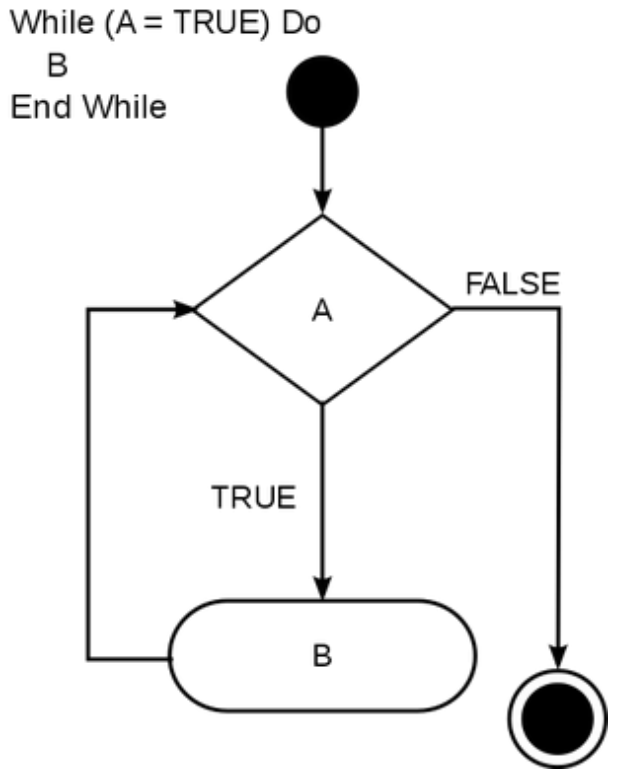
Selection

```
C:\dev\workspace_git\DataScienceFac
100

Process finished with exit code 0
```

Estruturas de Controle de Fluxo

- Estruturas de Repetição são utilizadas para instruir o programa a repetir um determinado conjunto de código enquanto uma determinada condição seja satisfatória
- São utilizadas também para iterar em coleções de dados ou listas
- Em Python, existem 2 tipos de estruturas de repetição:
  - for
  - while
- O for é mais utilizada na iteração de listas ou quando há um número conhecido de iterações a serem executadas
- O while é mais recomendado quando uma determinada condição seja satisfatória para a execução do trecho de código



Estruturas de Controle de Fluxo

Loops.py

```python
for letter in 'Python':
    print('The letter is: ', letter)


frutas = ['banana', 'maça', 'laranja']
for fruta in frutas:
    print(fruta)


comidas = ('Pizza', 'Arroz', 'Feijão')
for index in range(len(comidas)):
    print(comidas[index])

```

Loops.py

```python
count = 0
while count < 5:
    print(count)
    count = count + 1

```

Estruturas de Controle de Fluxo

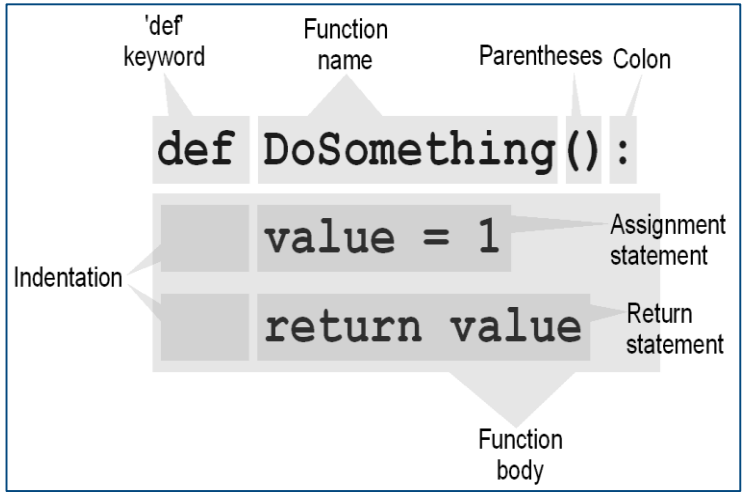
- Estruturas de Repetição oferecem instruções de controle em determinadas situações:
- **break** – permite a saida do fluxo antes da condição de finalização do laço ser atingida
- **continue** – Instrui o interpretador a passar para a próxima iteração imediatamente, ignorando os comandos subsequentes
- **pass** – permite que um determinado trecho de código seja escrito sintaticamente correto, porém em sua execução não será executada nenhuma instrução naquele momento

Funções

- Simplificadamente, uma função é um conjunto de instruções do qual pode-se dar um nome
- Funções são uma forma muito util de organizar o código
- Promovem o reaproveitamento, uma vez que um podem ser invocadas em diversos pontos do mesmo programa, sem a necessidade reescrita desse trecho de código
- A sintaxe de definição de uma função é a seguinte:

```
def NOME( PARÂMETROS ):
    COMANDOS
```

- Note a identação na parte de comandos, ela é fundamental para que o interpretador identifique quais são de fato, as instruções que pertencem à função



Mais sobre Funções

- Os parâmetros de uma função podem ser passados na mesma ordem em que foram definidos ou podem ser identificados através da função chamadora, assim não necessitando ser colocados na mesma ordem da definição da função
- Parâmetros de funções podem ter valores *default*, dessa forma não é necessário sua passagem como argumento (desde sejam os ultimos argumentos em ordem reversa)
- Funções suportam parâmetros com número indefinido de valores. Para isso é necessário que sejam o último parâmetro da função e o argumento seja precedido por um * (asterisco). O argumento será passado para a função como uma tupla de valores.
- Funcões podem retornar valores, através da instrução *return*. Não é necessário declarar que a função irá ter um retorno

Functions.py

```
1  def funcname(nome, idade, sexo, *outros):
2      print(type(nome), type(idade), type(sexo), type(outros))
3      print(nome, idade, sexo, outros)
4
5
6  funcname('nome', 25, 'm')
7  funcname('nome', 25, 'f', 'outros1', 'outros2')
8
9
```

Functions

```
C:\dev\workspace_git\DataScienceFacens\HelloWorldProject\ve
<class 'str'> <class 'int'> <class 'str'> <class 'tuple'>
nome 25 m ()
<class 'str'> <class 'int'> <class 'str'> <class 'tuple'>
nome 25 f ('outros1', 'outros2')
```

Operadores Aritméticos

Operador	Descrição	Exemplo
+	Adição	10 + 15 = 25
-	Subtração	25 – 15 = 10
*	Multiplicação	10 * 3 = 30
/	Divisão	30 / 4 = 7.5
//	Parte inteira da divisão	30 // 4 = 7
%	Módulo (Resto da divisão)	30 % 4 = 2
**	Exponenciação (Potência)	3**4 = 3 * 3 * 3 * 3 = 81

Operadores Relacionais

Operador	Descrição	Exemplo
==	Igual	10 > 15 é falso; 'Python' == 'python' é falso
!=	Diferente	5 != 7 é verdadeiro, 'Python' != 'python' é verdadeiro
>	Maior	5 > 5 é falso; 5 > 2 é verdadeiro
<	Menor	7 < 12 é verdadeiro; 7 < 5 é falso
>=	Maior ou igual	5 >= 5 é verdadeiro; 5 >= 6 é falso
<=	Menor ou igual	5 <= 5 é verdadeiro; 5 < 7 é verdadeiro

Operadores Lógicos

Operador	Descrição
and	Retorna True se os dois operandos forem True
or	Retorna True se um dos operandos for True
not	Negativa ou reverte a estado do operando

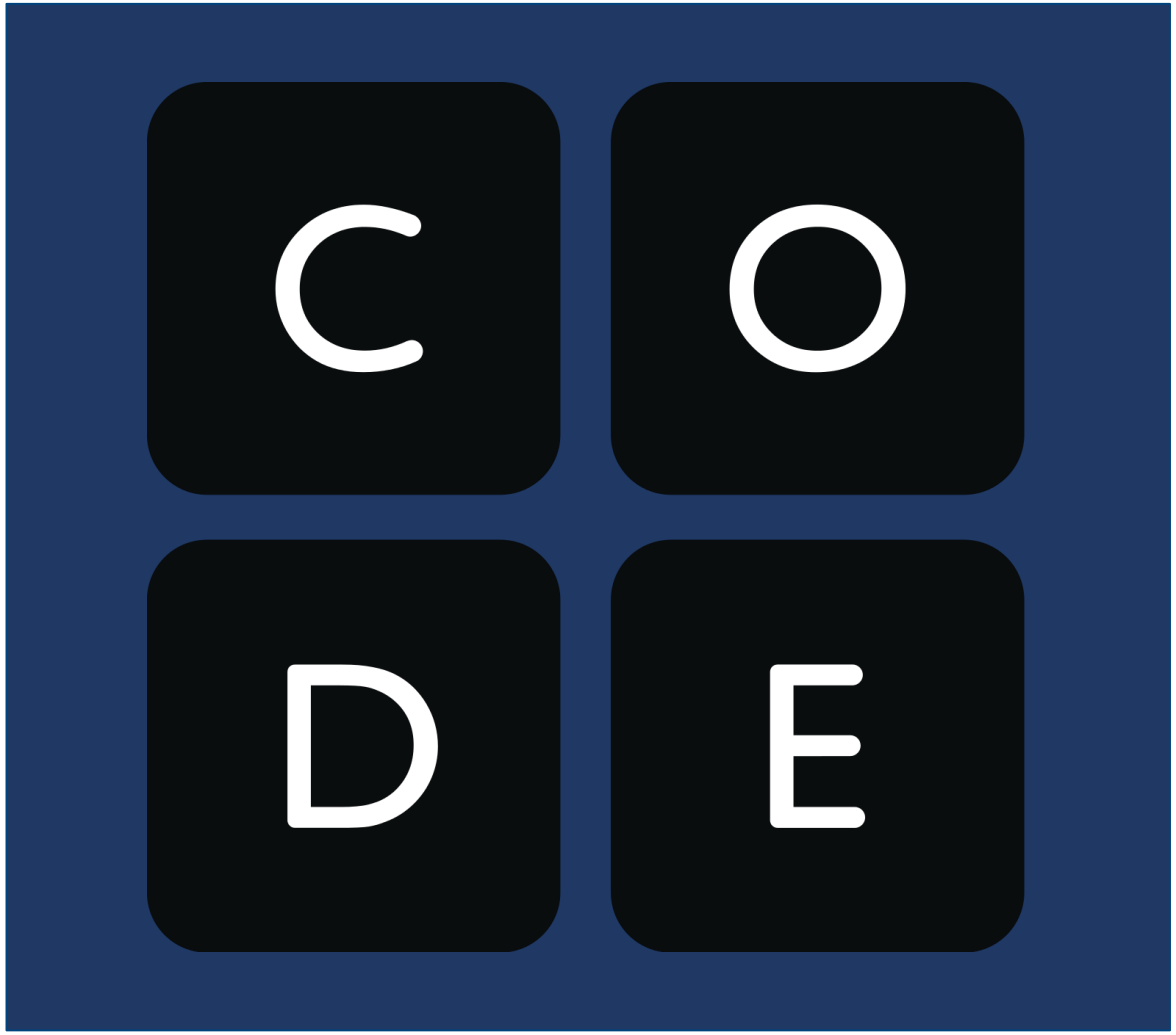
Operadores de Membro

Operador	Descrição
in	True se encontra a expressão pesquisada

Operadores de Identidade

Operador	Descrição
is	True se os operadores apontarem para o mesmo endereço de memória

Challenges Time





Exercicios

- Escreva uma função que receba três parâmetros (a, b e c) inteiros e retorne o maior valor entre os argumentos passados

- Escreva uma função que receba uma lista de números e retorne uma tupla com duas listas, a primeira contendo todos os numeros pares e a segunda contendo todos os numeros impares ordenados

Exercicios

- Escreva uma função que receba dois valores inteiros positivos, e retorne a soma da sequencia numérica inteira entre o primeiro e o segundo parâmetros, inclusive

- Dado um número inteiro não negativo (n), escreva duas funções sendo a primeira usando estruturas de repetição FOR e a segunda usando estruturas de repetição WHILE, que retornem o fatorial de n (n!)

Referências



- https://www.python.org/
- https://python.org.br/
- http://www.diveintopython.net/
- https://www.w3schools.com/python
- https://www.codecademy.com/tracks/python
- https://www.jetbrains.com/pycharm/



Facens

AQUI TEM ENGENHARIA